Coding & Solution

Date	30 June 2026
Team ID	xxxxxx
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution. The submission demonstrates working functionality, clean code practices, and alignment with defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/OptiCrop-Team/opticrop-smart-agriculture
Programming Language(s)	Python 3.10
Framework(s) Used	Flask (Backend API), Streamlit (Web UI), scikit-learn (ML Models)
Key Features Implemented	1. Crop Recommendation Engine (N,P,K + climate inputs → best crop) 2. Soil-Climate Suitability Checker 3. Fertilizer Optimization Module 4. Farmer Input Dashboard (Streamlit) 5. Flask REST API endpoints 6. Crop Compatibility Report Generator
Pending / Incomplete Features	1. Researcher Analytics Portal (charts & filters – In Progress) 2. Historical Yield Tracker (database integration – In Progress) 3. Multilingual UI support (Planned)
Setup / Run Instructions	1. Clone repo: git clone https://github.com/OptiCrop-Team/opticrop-smart-agriculture 2. Install dependencies: pip install -r requirements.txt 3. Train model: python train_model.py 4. Run API: python app.py 5. Run UI: streamlit run dashboard.py 6. Open browser at http://localhost:8501

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes

5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The OptiCrop codebase follows a clean MVC-like separation: model training scripts are isolated in /models/, Flask API routes are in /api/, and the Streamlit UI in /dashboard/. All ML functions are wrapped in reusable utility modules (crop_utils.py, soil_utils.py). The project uses a Kaggle Crop Recommendation Dataset (2200 records) and achieves 97.9% test accuracy with the Random Forest model. Unit tests are written using pytest and cover all critical prediction functions.